

CS 551 Systems Programming

Lecture 2: Enums and Pattern Matching

Where we left off

Lecture 1 ended with a guessing game that plays exactly one round and compares with `if/else if/else`.

Two things were missing:

1. A way to loop until the player is right.
2. A better way to express a three-way decision than a chain of `if` s.

Both are today. The second one is the interesting one, and it turns out to be the front door to a much bigger idea.

Today's path

1. **Enums.** Types that are one of several things.
2. `match`. Pattern matching, and why it beats `switch`.
3. **Expressions.** Why the semicolon is load-bearing in Rust.
4. **Control flow.** `if` as an expression, and the three loops.
5. **Conveniences.** `if let`, `let`-else, and friends.

1. Enums

Types that are one of several things

An enum names a fixed set of possibilities

```
1 enum Ordering {  
2     Less,  
3     Equal,  
4     Greater,  
5 }
```

This is a real type from the standard library, `std::cmp::Ordering`. A value of type `Ordering` is *exactly one of those three things*, and the compiler knows the list is complete.

That completeness is the whole point. It is what lets the compiler check that you handled every case, which a C `enum` (really just named integers) cannot do.

Variants are not types

This trips up everyone arriving from Java or C#.

`Ordering::Greater` is a **value** of type `Ordering`. It is not a type of its own.

```
1 use std::cmp::Ordering;
2
3 fn describe() -> Ordering::Greater { // does NOT compile: not a type
4     Ordering::Greater
5 }
```

```
error[E0573]: expected type, found variant `Ordering::Greater`
```

There is no type called `Ordering::Greater`, so that signature is meaningless. The function returns an `Ordering`.

CS 551 Hold onto this. `Some` is not a type either, and neither is `Err`. We will see both in a minute.

Enum variants can carry data

Variants are not limited to bare names. Each one can hold values, and different variants can hold different things:

```
1 enum Shape {  
2     Point,           // nothing  
3     Circle(f64),     // a radius  
4     Rect { w: f64, h: f64 }, // named fields  
5 }
```

This is the feature C enums do not have and the reason Rust enums are worth a lecture. A `Shape` is one of three shapes, *and* it carries exactly the data that shape needs, with no wasted space and no invalid combinations.

Languages call this a *sum type* or a *tagged union*. Rust just calls it an enum.

You have already used two

```
1 enum Option<T> {  
2     Some(T),  
3     None,  
4 }  
5 enum Result<T, E> {  
6     Ok(T),  
7     Err(E),  
8 }
```

You have already used two

```
1 enum Option<T> {  
2     Some(T),  
3     None,  
4 }  
5 enum Result<T, E> {  
6     Ok(T),  
7     Err(E),  
8 }
```

The <T> is a **type parameter**: one enum, any type inside. Lecture 9 is about these

Both of the types you met in Lecture 1 are enums. `Option<T>` is "a `T`, or nothing," which is how Rust says what other languages say with `null`. `Result<T, E>` is "a `T`, or an error," which is how it says what they say with exceptions.

Neither is special. Neither is built into the language.

2. `match`

How you take an enum apart

The guessing game comparison

```
1 use std::cmp::Ordering;
2
3 let guess = 50;
4 let secret_number = 42;
5
6 match guess.cmp(&secret_number) {
7     Ordering::Greater => println!("You guessed too high!"),
8     Ordering::Less => println!("You guessed too low!"),
9     Ordering::Equal => println!("You got it right!"),
10 }
```

The guessing game comparison

```
1 use std::cmp::Ordering;
2
3 let guess = 50;
4 let secret_number = 42;
5
6 match guess.cmp(&secret_number) {
7     Ordering::Greater => println!(
8     Ordering::Less => println!("Yo
9     Ordering::Equal => println!("You got it right!"),
10 }
```

`cmp` returns an **Ordering**: exactly one of three values, and the compiler knows all three

The guessing game comparison

```
1 use std::cmp::Ordering;
2
3 let guess = 50;
4 let secret_number = 42;
5
6 match guess.cmp(&secret_number) {
7     Ordering::Greater => println!("You guessed too high!"),
8     Ordering::Less => println!("You guessed too low!"),
9     Ordering::Equal => println!("You got it right!"),
10 }
```

The **match** contains three **arms**. Each is a pattern, a fat arrow, and an expression

The guessing game comparison

```
1 use std::cmp::Ordering;
2
3 let guess = 50;
4 let secret_number = 42;
5
6 match guess.cmp(&secret_number) {
7     Ordering::Greater => println!("You guessed too high!"),
8     Ordering::Less => println!("You guessed too low!"),
9     Ordering::Equal => println!("You got it right!"),
10 }
```

Left of the `=>` is the **pattern**
the value is tested against

The guessing game comparison

```
1 use std::cmp::Ordering;
2
3 let guess = 50;
4 let secret_number = 42;
5
6 match guess.cmp(&secret_number) {
7     Ordering::Greater => println!("You guessed too high!"),
8     Ordering::Less => println!("You guessed too low!")
9     Ordering::Equal => println!("You got it right!"),
10 }
```

Right of it is the **expression** that runs when the pattern matches

The guessing game comparison

```
1 use std::cmp::Ordering;
2
3 let guess = 50;
4 let secret_number = 42;
5
6 match guess.cmp(&secret_number) {
7     Ordering::Greater => println!("You guessed too high!")
8     Ordering::Less => println!("You guessed too low!"),
9     Ordering::Equal => println!("You got it right!"),
10 }
```

Note that those are **commas**, not semicolons. What is on the right of the `=>` is an expression, not a statement

**That distinction
runs through the
whole language.**

match is exhaustive

Leave out a case and the program does not compile:

```
1 use std::cmp::Ordering;
2
3 let n = 5.cmp(&6);
4
5 match n {
6     Ordering::Greater => println!("high"),
7     Ordering::Less => println!("low"),
8 }
```

```
error[E0004]: non-exhaustive patterns: `Ordering::Equal` not covered
```

This is the payoff for enums having a known, closed list of variants. **The compiler will not let you forget a case.** Add a variant to an enum later and every `match` that needs updating becomes a compile error rather than a silent bug.

The catch-all patterns

```
1 let n = 7;
2
3 match n {
4     1 => println!("one"),
5     2 => println!("two"),
6     other => println!("something else: {other}"),
7 }
8
9 match n {
10     1 => println!("one"),
11     _ => println!("don't care"),
12 }
```

The catch-all patterns

```
1 let n = 7;
2
3 match n {
4     1 => println!("one"),
5     2 => println!("two"),
6     other => println!("something else: {other}"),
7 }
8
9 match n {
10    1 => println!("one"),
11    _ => println!("don't care"),
12 }
```

A **binding**. It catches everything and hands you the value

A **wildcard**. It catches everything and throws the value away

Reach for these sparingly on enums. A `_` arm is exactly what turns "you forgot a case" back into a silent bug.

Patterns destructure

Matching does not just test which variant it is, it takes the value apart:

```
1 enum Shape {
2     Point,
3     Circle(f64),
4     Rect { w: f64, h: f64 },
5 }
6
7 fn area(s: &Shape) -> f64 {
8     match s {
9         Shape::Point => 0.0,
10        Shape::Circle(r) => 3.14159 * r * r,
11        Shape::Rect { w, h } => w * h,
12    }
13 }
14
15 fn main() {
16     println!("{}", area(&Shape::Rect { w: 3.0, h: 4.0 }));
17 }
```

`r`, `w`, and `h` are new bindings, pulled out of the value by the pattern.

match is an *expression*

Every arm produces a value, and the whole `match` evaluates to it:

```
1 use std::cmp::Ordering;
2
3 let guess = 50;
4 let secret = 42;
5 let message = match guess.cmp(&secret) {
6     Ordering::Greater => "too high",
7     Ordering::Less => "too low",
8     Ordering::Equal => "correct",
9 };
10
11 println!("{message}");
```

Note the semicolon after the closing brace: `let` is a *statement*.

Every arm must produce the **same type**.

Arms can be blocks

An arm is one expression, but a block is one expression:

```
1  let n = 3;
2
3  let label = match n {
4      0 => "zero",
5      _ => {
6          let doubled = n * 2;
7          println!("doubling {n} to {doubled}");
8          "nonzero"
9      }
10 };
11
12 println!("{label}");
```

The block's value is its last expression, with no semicolon. That rule is next.

Recap

1. **An enum names a fixed, closed set of possibilities**, and the compiler knows the list is complete.
2. **Variants are not types.** `Ordering::Greater` is a value of type `Ordering`, and so is `Some(5)` a value of type `Option<i32>`.
3. **Variants carry data**, different data per variant, with no wasted space and no invalid combinations. This is what a C enum cannot do.
4. `Option<T>` and `Result<T, E>` are ordinary enums. Neither is built into the language.
5. `match` is exhaustive. Forgetting a case is a compile error, and adding a variant later turns every stale `match` into one.
6. **Patterns destructure.** Matching pulls the data out of the variant and binds it.
7. `_` is a wildcard, and a liability on enums: it turns "you forgot a case" back into a silent bug.

Exercises

1. Write an enum for a traffic light and a function returning how long each colour lasts. Add a fourth variant and watch which `match` expressions stop compiling.
2. Write `fn describe(n: Option<i32>) -> String` using `match`. Then rewrite it without `match` and say which reads better.
3. Take a `match` with a `_` arm over an enum you control. Add a variant. Explain, in one sentence, what the compiler did not tell you.
4. Write a `Shape` enum with three variants carrying different data, and an `area` function over it. Compare it to how you would do the same thing in a language with inheritance.
5. `std::cmp::Ordering` has three variants. Find its definition in the standard library source and confirm there is nothing special about it.

Reading

- **The Rust Programming Language**, chapter 6 (Enums and Pattern Matching), and chapter 18 for patterns in depth.
- **Rust by Example**, the sections on enums and `match`.
- **The `std::option` and `std::result` module docs**. Skim the method lists once; a great deal of what you would write by hand already exists.

Next: Lecture 3

Expressions and Control Flow

Why the semicolon is load-bearing, and the three loops.